

System Programming Project 4

담당 교수 : 김영재

이름 : 이충인

학번 : 20221609

1. 개발 목표

- 해당 프로젝트에서 구현할 내용을 간략히 서술.

Dynamic memory allocator를 구현하는 프로젝트이다. 정확하고 효율적이며 빠른 malloc, free, realloc 함수를 직접 구현한다.

2. 개발 범위 및 내용

A. 개발 내용

`int mm_init(void)` : mm_malloc, mm_realloc, mm_free 를 호출하기 전에 초기 힙 영역을 할당하는 등 필요한 초기화 작업을 수행한다. 초기화 성공 시 0 을 반환하고 오류 발생 시 -1 을 반환한다.

`void *mm_malloc(size_t size)` : libc malloc 과 마찬가지로, 8byte aligned 된 최소 payload 만큼 할당된 block 의 포인터를 반환한다.

`void mm_free(void *bp)` : ptr 가 가리키는 block 을 해제한다. 이전에 mm_malloc 또는 mm_realloc 을 통해 할당된 ptr 에 대해서만 해제 작업을 수행한다.

`void *mm_realloc(void *ptr, size_t size)` : ptr 이 NULL 이면 mm_malloc 과 동일한 작업을 수행한다. size 가 0 이면 mm_free 와 동일한 작업을 수행한다. ptr 이 NULL 이 아닌 경우, 요청한 size 로 memory block 의 크기를 바꾸고 새로운 주소를 반환한다.

B. 개발 방법

- Description of subroutines, structs, any global variables

```
static void *heap_listp; // start of heap
static void *free_listp; // first free block
```

heap_listp는 힙 영역의 시작 주소를 나타낸다. free_listp는 explicit free list의 첫 번째 메모리 블록의 주소를 나타낸다.

```
#define NEXT_FREE(bp) (*(char **)(bp))
#define PREV_FREE(bp) (*(char **)(bp + WSIZE))
```

위 두 매크로는 explicit free list 를 구현하기 위함이다. free 된 memory block 들을 리스트로 저장하기 위해, NEXT_FREE(bp) 매크로는 현재 블록(bp)의 다음 블록 주소를, PREV_FREE(bp)는 이전 블록 주소를 의미한다.

```
int mm_init(void);
```

`free_listp`를 `NULL`로 초기화하고, `mem_sbrk` 함수를 통해 초기 힙 공간을 할당한다. 초기 힙 영역에서 프롤로그, 에필로그 `header`, `footer`를 설정하여, 메모리 할당 시 힙 영역을 벗어나는 것을 방지한다. `heap_listp`를 프롤로그 푸터로 주소를 이동시켜, 해당 주소의 영역부터 메모리 할당을 할 수 있도록 한다. `extend_heap` 함수를 호출하여 힙 영역을 확장시킨다. 모든 초기화 과정이 성공적으로 이루어지면 `0`을 반환하고, 오류가 발생했다면 `-1`을 반환한다.

```
void *mm_malloc(size_t size);
```

`size`는 할당하려는 메모리의 `byte` 수를 나타내는 매개변수이다. `asize`는 `8byte alignment`를 충족하기 위해 적절한 크기를 계산한 메모리 블록의 크기를 저장하는 변수이다. `find_fit` 함수를 호출하여 `free list`에서 적절한 크기의 메모리 블록을 찾는다. 만약 `free list`에서 적절한 블록을 찾지 못했을 경우, `extend_heap` 함수를 통해 힙 영역을 확장한다. `place` 함수를 통해 탐색한 적절한 블록(`bp`)에 실제로 메모리를 할당한다.

```
void mm_free(void *bp);
```

`bp`는 해제하려는 메모리 블록을 가리키는 포인터이다. 해당 블록의 헤더와 푸터를 설정하여 `status`를 `0`으로 변경한다. `insert` 함수를 호출하여 해제된 메모리 블록을 `free list`에 삽입하여 이후 해당 블록을 재사용할 수 있도록 한다. `coalesce` 함수를 호출하여 경우에 따라 해제된 블록의 주변 `free` 블록과 합친다. 이를 통해 메모리 단편화를 감소시킨다.

```
void *mm_realloc(void *ptr, size_t size);
```

`ptr`가 가리키는 메모리 블록의 크기를 `size`로 변경한다. 만약 `size`가 `0`이면 `ptr`가 가리키는 메모리를 해제하고 `NULL`을 반환한다. `ptr`가 가리키는 메모리 블록의 크기가 `8byte alignment`에 따라 조정된 `size(assize)`보다 크거나 같으면 `ptr`를 그대로 반환한다. `ptr` 다음 블록이 비어 있고 해당 블록을 포함해 `asize`를 만족할 수 있는 경우, 인접 블록을 결합하여 할당한다. 위 경우 모두 해당되지 않으면, `size`만큼 새로 메모리를 할당하여 기존 데이터를 복사하고 기존 메모리 블록을 해제한다. 최종적으로 새로 할당한 메모리 블록의 포인터를 반환한다.

```
static void *extend_heap(size_t words);
```

초기 힙 영역의 크기를 확장하거나 메모리 블록을 추가로 할당하는 함수이다. `mem_sbrk` 함수를 통해 힙 영역의 끝에 새로운 메모리를 할당한다. 새로 할당된 메모리 블록에 대해 헤더와 푸터를 설정하고 에필로그 블록을 설정하여 빈 메모리 블록의 끝을 표시한다. `insert` 함수를 통해 확장된 메모리 블록을 `free list`에 삽입한다. 필요한 경우 이전의 빈 블록과 합치기 위해 최종적으로 `coalesce` 함수를 호출한다. 이에 따른 메모리 블록의 포인터를 반환한다.

```
static char *extend_heap_realloc(char *bp, size_t asize, int *left);
```

`mm_realloc` 함수에서 호출되어, 메모리 블록을 확장하는 함수이다. `left` 변수에 현재 메모리 블록(`bp`)과 다음 메모리 블록을 합쳤을 때 `assize`를 제외하고 남은 공간을 저장한다. `left`가 음수인 경우 즉, 필요한 `assize`보다 작은 메모리 블록 크기의 합이 작은 경우 추가 메모리를 요청하여 합을 확장한다. 확장된 메모리 블록의 포인터를 반환한다. `assize`보다 블록 크기가 합이 큰 경우 포인터를 그대로 반환한다.

```
static void *coalesce(void *bp);
```

메모리 블록이 해제되었을 때 인접한 빈 블록들을 합치는 작업을 통해 **external fragmentation**을 줄여 메모리 관리 효율성을 높이고자 한다. 인접한 블록의 상태(**free / allocated**)에 따라 경우를 네 가지로 나누어, 인접한 블록이 **free**인 경우 요청된 블록과 합친다. 각 경우에서 **delete** 함수를 통해 **free list**에서 제거를 하고 합병된 새로운 빈 블록을 **insert** 함수를 통해 **free list**를 통해 다시 삽입하고, 헤더와 푸터를 업데이트한다.

```
static void *find_fit(size_t asize);
```

메모리 할당 시 요청된 크기(**asize**)에 맞는 빈 메모리 블록을 찾는다. **first-fit** 방식으로 **free list**를 **free_listp**(**list**의 첫 번째 블록)부터 순회하며 **asize**에 맞는 빈 블록을 찾는다. 조건에 맞는 블록을 찾는 경우 해당 블록 포인터를 반환하고 그렇지 못한 경우 **bp = NULL**을 반환한다.

```
static void *place(void *bp, size_t asize);
```

find_fit 함수에서 반환한 메모리 블록(**bp**)에 실제로 할당 작업을 수행하는 함수이다. 먼저 **delete(bp)**를 호출하여 **bp**를 **free list**에서 제거한다. **free_left** 변수는 **bp**의 블록 크기(**csize**)에서 요청한 **asize**을 제외한 남은 공간의 크기를 나타낸다. **free_left**가 최소 할당 크기인 **16byte**보다 작은 경우, 전체 블록을 할당한다. **asize**를 할당할 만큼 공간이 있는 경우 해당 블록을 **asize**, **free_left**의 크기로 **split**하여 할당된 공간의 헤더와 푸터를 업데이트하고, **free_left**만큼의 새로운 빈 블록을 **insert** 함수를 통해 **free list**에 추가한다. **asize**가 **24byte** 이상인 경우 **free_left**, **asize**의 크기로 **split**하여 할당된 공간의 헤더와 푸터를 업데이트한다. 이후 **free_left**만큼의 새로운 빈 블록을 **insert** 함수를 통해 **free list**에 추가한다.

```
static void insert(size_t size, char *new);
```

free_listp(**free list**의 첫 번째 블록)부터 리스트를 순회하면서 새로 삽입하려는 블록(**new**)의 크기인 **size**보다 큰 첫 번째 블록(**cur**)을 찾는다. 새로운 블록을 첫 번째 블록으로 삽입하는 경우, **free_listp**를 **new**로 설정하고 **new**의 **PREV_FREE**를 **NULL**로 설정한다. 이때 현재 리스트에 블록이 하나만 있는 경우 **new**의 **NEXT_FREE**를 **NULL**로 설정하고, 아닌 경우 **new**와 **cur**을 순서대로 연결해준다. 새로운 블록을 중간이나 마지막에 삽입하는 경우, **prev**와 **new**를 순서대로 연결해준다. 이후 마지막에 삽입되는 경우 **NEXT_FREE(new)**를 **NULL**로 설정한다.

```
static void delete(char *del);
```

del 블록을 **free list**에서 삭제한다. 첫 번째 블록을 삭제하는 경우, **free_listp**를 **NEXT_FREE(del)**로 설정하고, 첫 번째 블록의 **PREV_FREE**를 **NULL**로 설정한다. 그렇지 않은 경우, **PREV_FREE(del)**의 **NEXT_FREE**를 **NEXT_FREE(del)**로 설정하고, 다음 블록이 존재할 경우 해당 블록의 **PREV_FREE**를 **PREV_FREE(del)**로 설정한다.

./mdriver -v를 실행한 결과는 다음과 같다.

[20221609]::NAME: ChoongIn Lee, Email Address: hooi0318@sogang.ac.kr
Using default tracefiles in ./tracefiles/
Measuring performance with gettimeofday().

Results for mm malloc:

trace	valid	util	ops	secs	Kops
0	yes	99%	5694	0.000478	11917
1	yes	99%	5848	0.000359	16294
2	yes	99%	6648	0.000861	7721
3	yes	99%	5380	0.000373	14416
4	yes	97%	14400	0.000207	69700
5	yes	96%	4800	0.009925	484
6	yes	95%	4800	0.009434	509
7	yes	61%	12000	0.078217	153
8	yes	88%	24000	0.021406	1121
9	yes	83%	14401	0.000233	61701
10	yes	85%	14401	0.000136	105968
Total		91%	112372	0.121630	924

Perf index = 55 (util) + 40 (thru) = 95/100

- Design of your allocator with any type of figures

